



Gabriel dos Santos Silva Coelho

RESENHA

Artigo 10 - Design By Contract

Meyer começa seu argumento falando sobre um problema que continua muito presente no desenvolvimento de software: a confiabilidade dos sistemas. Ele destaca que existe uma distância considerável entre a teoria, baseada em verificação formal, e a prática do dia a dia, onde fatores importantes como correção e robustez acabam ficando em segundo plano diante da flexibilidade proporcionada pela orientação a objetos. Como forma de resolver essa questão, Meyer apresenta a metáfora do contrato, funcionando como um acordo entre cliente (quem chama a rotina) e fornecedor (quem executa), definindo de maneira clara as responsabilidades e os benefícios de cada lado.

O ponto central técnico do Design by Contract (DbC) está nas assertivas. Meyer explica três tipos principais: as pré-condições, que representam os requisitos que o cliente deve cumprir para que a chamada seja válida; as pós-condições, que são as garantias entregues pelo fornecedor após a execução, desde que a pré-condição tenha sido respeitada; e os invariantes de classe, responsáveis por manter a consistência dos objetos em estados observáveis. Uma das coisas mais interessantes do texto é a crítica feita à programação defensiva. Para Meyer, quando cliente e fornecedor fazem verificações diretas, o sistema se torna mais complexo e até menos confiável. Ao definir claramente no contrato quem é responsável por cada condição, a arquitetura acaba ficando mais organizada, simples e limpa. Ao aplicar o Design by Contract à herança, Meyer deixa mais claros os riscos envolvidos na ligação dinâmica. Ele compara a redefinição de métodos a uma espécie de subcontratação: a classe filha assume a responsabilidade de cumprir o contrato definido

pela classe pai. Para que isso funcione corretamente, o “subcontratado” não pode exigir mais do que o contrato original nem entregar menos do que foi prometido. A partir dessa ideia, Meyer define a regra da redefinição: a pré-condição só pode ser enfraquecida, permitindo aceitar mais situações de entrada, enquanto a pós-condição só pode ser fortalecida, garantindo resultados melhores ou mais específicos ao cliente.

No artigo, Meyer também faz uma crítica bastante forte aos mecanismos de exceção presentes em linguagens como Ada e CLU, principalmente porque eles permitem que uma rotina falhe “silenciosamente” ou termine sem realmente cumprir o que prometeu. Para Meyer, isso compromete a confiabilidade do software. Ele defende regras mais rígidas: ou a rotina cumpre o contrato estabelecido, ou então falha de forma explícita e informa o chamador sobre o problema. No modelo Eiffel, essa ideia é aplicada por meio das cláusulas *rescue* e *retry*.

Por fim, o autor encerra o texto com um verdadeiro “apelo pelas funções parciais”, questionando a ideia de que rotinas precisam ser capazes de lidar com qualquer tipo de entrada. Para ele, a robustez de um sistema não surge da tentativa de prever todos os erros possíveis, mas sim da clareza na documentação e da definição precisa das responsabilidades de cada parte. Nesse contexto, o Design by Contract transforma as assertivas, que antes funcionavam apenas como comentários informais, em elementos fundamentais da especificação e do próprio design do software